

```

timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"

module tb_branch_decision();

    // Inputs
    reg [31:0] i_read_data1;
    reg [31:0] i_read_data2;
    reg [2:0] i_Func3;
    reg [4:0] i_Funct5;
    reg [6:0] i_opcode;

    // Outputs
    wire o_Branch;
    wire [31:0] o_AMO_result;

    // Instantiate the DUT (Device Under Test)
    branch_decision uut (
        .i_read_data1(i_read_data1),
        .i_read_data2(i_read_data2),
        .i_Func3(i_Func3),
        .i_Funct5(i_Funct5),
        .i_opcode(i_opcode),
        .o_Branch(o_Branch),
        .o_AMO_result(o_AMO_result)
    );

    initial begin
        // Test Branch Instructions
        $display("Testing Branch Instructions...");

        i_opcode = `B; // Branch opcode

        // Test BEQ (Equal)
        i_Func3 = `BEQ; i_read_data1 = 32'd10; i_read_data2 = 32'd10;
        #10 $display("BEQ: o_Branch = %b (Expected: 1)", o_Branch);

        // Test BNE (Not Equal)
        i_Func3 = `BNE; i_read_data1 = 32'd10; i_read_data2 = 32'd20;
        #10 $display("BNE: o_Branch = %b (Expected: 1)", o_Branch);

        // Test BLT (Signed Less Than)
        i_Func3 = `BLT; i_read_data1 = -5; i_read_data2 = 10;
        #10 $display("BLT: o_Branch = %b (Expected: 1)", o_Branch);
    end
endmodule

```

```

// Test BGE (Signed Greater or Equal)
i_Func3 = `BGE; i_read_data1 = 10; i_read_data2 = -5;
#10 $display("BGE: o_Branch = %b (Expected: 1)", o_Branch);

// Test BLTU (Unsigned Less Than)
i_Func3 = `BLTU; i_read_data1 = 32'h00000005; i_read_data2 = 32'h00000010;
#10 $display("BLTU: o_Branch = %b (Expected: 1)", o_Branch);

// Test BGEU (Unsigned Greater or Equal)
i_Func3 = `BGEU; i_read_data1 = 32'h00000010; i_read_data2 = 32'h00000005;
#10 $display("BGEU: o_Branch = %b (Expected: 1)", o_Branch);

// Test Atomic Operations
$display("\nTesting Atomic Operations...");

i_opcode = `AMO; // Atomic opcode

// Test AMO_ADD
i_Funct5 = `AMO_ADD; i_read_data1 = 10; i_read_data2 = 20;
#10 $display("AMO_ADD: o_AMO_result = %d (Expected: 30)", o_AMO_result);

// Test AMO_SWAP
i_Funct5 = `AMO_SWAP; i_read_data1 = 100; i_read_data2 = 200;
#10 $display("AMO_SWAP: o_AMO_result = %d (Expected: 200)", o_AMO_result);

// Test AMO_XOR
i_Funct5 = `AMO_XOR; i_read_data1 = 8'hF0; i_read_data2 = 8'h0F;
#10 $display("AMO_XOR: o_AMO_result = %h (Expected: FF)", o_AMO_result);

// Test AMO_AND
i_Funct5 = `AMO_AND; i_read_data1 = 8'hFF; i_read_data2 = 8'hF0;
#10 $display("AMO_AND: o_AMO_result = %h (Expected: F0)", o_AMO_result);

// Test AMO_OR
i_Funct5 = `AMO_OR; i_read_data1 = 8'h0F; i_read_data2 = 8'hF0;
#10 $display("AMO_OR: o_AMO_result = %h (Expected: FF)", o_AMO_result);

// Test AMO_MIN (Signed)
i_Funct5 = `AMO_MIN; i_read_data1 = -10; i_read_data2 = 5;
#10 $display("AMO_MIN: o_AMO_result = %d (Expected: -10)", o_AMO_result);

// Test AMO_MAX (Signed)
i_Funct5 = `AMO_MAX; i_read_data1 = -10; i_read_data2 = 5;
#10 $display("AMO_MAX: o_AMO_result = %d (Expected: 5)", o_AMO_result);

// Test AMO_MINU (Unsigned)

```

```
i_Funct5 = `AMO_MINU; i_read_data1 = 10; i_read_data2 = 5;
#10 $display("AMO_MINU: o_AMO_result = %d (Expected: 5)", o_AMO_result);

// Test AMO_MAXU (Unsigned)
i_Funct5 = `AMO_MAXU; i_read_data1 = 10; i_read_data2 = 5;
#10 $display("AMO_MAXU: o_AMO_result = %d (Expected: 10)", o_AMO_result);

// End simulation
$display("\nTestbench Completed!");
$finish;
end
```

```
endmodule
```